COLLEGE OF ENGINEERING TRIVANDRUM
THIRUVANANTHAPURAM – 695016



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Fifth Semester

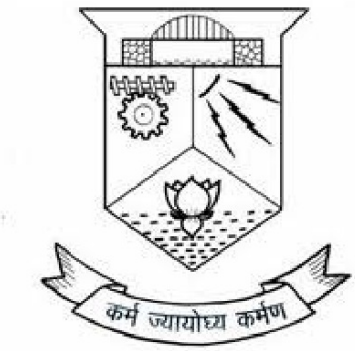
PCCSL508: MACHINE LEARNING LAB

Name : Woitiwe Simson
KTU ID : TVE24CS140

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

2026-2027

COLLEGE OF ENGINEERING TRIVANDRUM
THIRUVANANTHAPURAM – 695016



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Fifth Semester

PCCSL508: MACHINE LEARNING LAB

Certified that this is a bonafide record of work done by Woitiwe Simson (TVE24CS140) of S5 CSE (Batch-2) class in the Machine Learning Lab during the year 2026-2027.

External Examiner

Faculty Member in charge

Contents

Si.No.	Experiment Title	Page Number
1	Linear Regression	1
2	Polynomial Regression	5
3	Ridge and Lasso Regression	8
4	Logistic Regression	11
5	Naive Bayes Classifier	14
6	Logistic Regression with Regularization	17
7	MLE and MAP	20
8	K-Nearest Neighbors	23
9	Decision Tree using ID3	26
10	Support Vector Machines	29
11	SVM Kernels	32
12	Multilayer Feed-Forward Networks	35
13	Neural Networks	38
14	Hierarchical and Partitional Clustering	41
15	K-Means Clustering	44
16	Bootstrapping and Cross-Validation	47
17	Bagging and Boosting	50
18	Bias-Variance Tradeoff	53

Experiment 1 **Date: 16/07/2026**

Linear Regression

Aim

Implement linear regression with one variable on the California Housing dataset to predict housing prices based on a single feature (e.g., the average number of rooms per dwelling). Tasks:

- Load and preprocess the datase.
- Implement linear regression using both gradient descent and the normal equation.
- Evaluate the model performance using metrics such as Mean Squared Error (MSE) and R-squared.
- Visualize the fitted line along with the data points.

Algorithm

1. **Load and Extract Data:** Load the California Housing dataset, select a single feature (e.g., `AveRooms`), and extract the target (`MedHouseVal`).
2. **Preprocess Data:** Add a column of ones to the feature matrix to account for the intercept term and split the data into training and testing sets.
3. **Implement Gradient Descent:** Initialize parameters, then iteratively compute predictions, calculate Mean Squared Error, find gradients, and update parameters using a chosen learning rate.
4. **Implement Normal Equation:** Compute the optimal weights analytically using the closed-form formula: $\theta = (X^T X)^{-1} X^T y$.
5. **Evaluate Models:** Compute performance metrics (MSE and R^2 score) for both the gradient descent and normal equation models.
6. **Visualize Results:** Generate a scatter plot of the data points and overlay the fitted regression lines derived from both methods.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def gradientDescent(X, y, learningRate=0.01, maxIterations=5000, epsilon
    =1e-6):
    m = len(y)
    theta = np.zeros((X.shape[1], 1))
```

```
    costHistory = []

    for _ in range(maxIterations):
        predictions = X @ theta
        errors = predictions - y
        gradient = (learningRate / m) * (X.T @ errors)
        newTheta = theta - gradient

        theta_difference = np.linalg.norm(newTheta - theta)
        theta = newTheta

        cost = (1 / (2 * m)) * np.sum(errors ** 2)
        costHistory.append(cost)

        if theta_difference < epsilon:
            break
    return theta, costHistory

df = pd.read_csv("Datasets/housing.csv")

X = df[["MedInc"]].values
y = df[["MedHouseVal"]].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_bias = np.c_[np.ones((X_train_scaled.shape[0], 1)),
   X_train_scaled]
X_test_bias = np.c_[np.ones((X_test_scaled.shape[0], 1)), X_test_scaled]

theta_normal = np.linalg.inv(X_train_bias.T @ X_train_bias) @
   X_train_bias.T @ y_train
theta_gd, costHistory = gradientDescent(X_train_bias, y_train)
iterations = len(costHistory)

y_predicted_normal = X_test_bias @ theta_normal
y_predicted_gd = X_test_bias @ theta_gd

mse_normal = mean_squared_error(y_test, y_predicted_normal)
mse_gd = mean_squared_error(y_test, y_predicted_gd)

r2_normal = r2_score(y_test, y_predicted_normal)
r2_gd = r2_score(y_test, y_predicted_gd)

print(f"Gradient Descent converged after {iterations} iterations\n")

print(f"Errors for Gradient Descent: \nMSE = {mse_gd: .4f} \nR2 = {r2_gd
   : .4f}\n")
print(f"Errors for Normal Equation: \nMSE = {mse_normal: .4f} \nR2 = {
   r2_normal: .4f}\n")

plt.figure(figsize=(14, 5))
sample_size = min(200, len(X_test))
```

```
plt.scatter(X_test[:sample_size], y_test[:sample_size], alpha=0.5, color
    ='red')

xline = np.linspace(X_test.min(), X_test.max(), 100).reshape(-1,1)
xline_scaled = scaler.transform(xline)
xline_bias = np.c_[np.ones((xline_scaled.shape[0], 1)), xline_scaled]

yline_normal = xline_bias @ theta_normal
yline_gd = xline_bias @ theta_gd

plt.plot(xline, yline_normal, color='black', linewidth=3, label='Normal
    Equation')
plt.plot(xline, yline_gd, color='blue', linewidth=2, linestyle=':',
    label='Gradient Descent')

plt.xlabel("MedInc")
plt.ylabel("MedHouseVal")

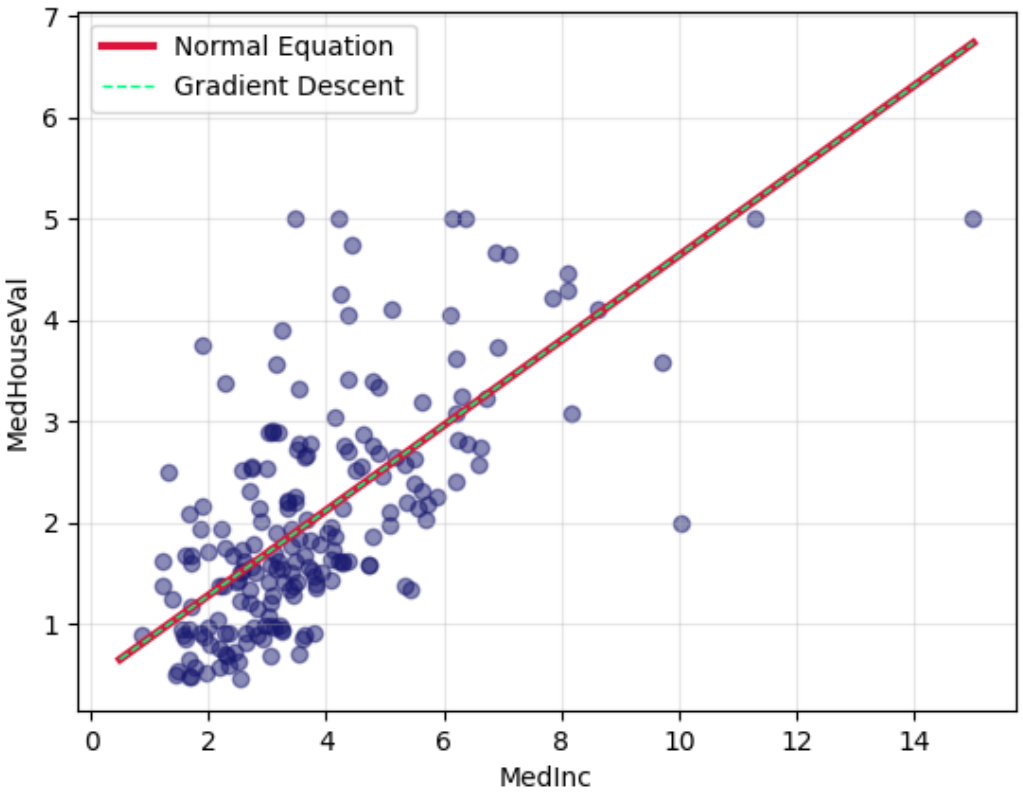
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Output

```
Gradient Descent converged after 997 iterations

Errors for Gradient Descent:
MSE =  0.7091
R2 =  0.4589

Errors for Normal Equation:
MSE =  0.7091
R2 =  0.4589
```



Result

The linear regression model was implemented successfully using both Gradient Descent and the Normal Equation, and the fitted regression line was also obtained

Experiment 2 **Date: 16/07/2026**

Polynomial Regression

Aim

Implement polynomial regression on the Auto MPG dataset to predict miles per gallon (MPG) based on engine displacement. Compare polynomial regression results with linear regression.
Tasks:

- Load and preprocess the dataset.
- Implement polynomial regression of varying degrees.
- Compare the polynomial regression models with linear regression using metrics such as MSE and R-squared.
- Visualize the polynomial fit.

Algorithm

1. **Load and Preprocess Data:** Load the Auto MPG dataset, extract engine displacement as the feature and MPG as the target, handle any missing values, and split the data into training and testing sets.
2. **Implement Linear Regression:** Fit a baseline simple linear regression model using the displacement feature.
3. **Implement Polynomial Regression:** Generate polynomial feature expansions for varying degrees and fit regression models on these transformed features.
4. **Evaluate and Compare Models:** Compute performance metrics (MSE and R^2 score) to compare the linear model with the different polynomial regression models.
5. **Visualize the Fits:** Generate a scatter plot of the data points and overlay the curves fitted by both the linear and polynomial regression models.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Datasets/autompg.csv")
df = df[['displacement', 'class']]
X = df[['displacement']].values
y = df[['class']].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size
    =0.2, random_state=42)

linearModel = LinearRegression()
linearModel.fit(X_train, y_train)
y_prediction_linear = linearModel.predict(X_test)

print("Errors for Linear Regression")
print(f"MSE = {mean_squared_error(y_test, y_prediction_linear): .4f} \
    nR2 = {r2_score(y_test, y_prediction_linear): .4f}\n")

degrees = [2, 3, 4]
models = {}
for deg in degrees:
    poly = PolynomialFeatures(degree=deg)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print(f"Errors for Polynomial Regression (degree={deg})")
    print(f"MSE = {mse: .4f} \nR2 = {r2: .4f}\n")

    models[deg] = (poly, model)

x_plot = np.linspace(X.min(), X.max(), 500).reshape(-1,1)

plt.figure(figsize=(10,6))
plt.scatter(X, y, color='blue', alpha=0.5, label='Data')

plt.plot(x_plot, linearModel.predict(x_plot), color='red', linewidth=2,
    label='Linear Regression')

colors = ['green', 'purple', 'orange']
for degree, color in zip(degrees, colors):
    poly, model = models[degree]

    y_curve = model.predict(poly.transform(x_plot))

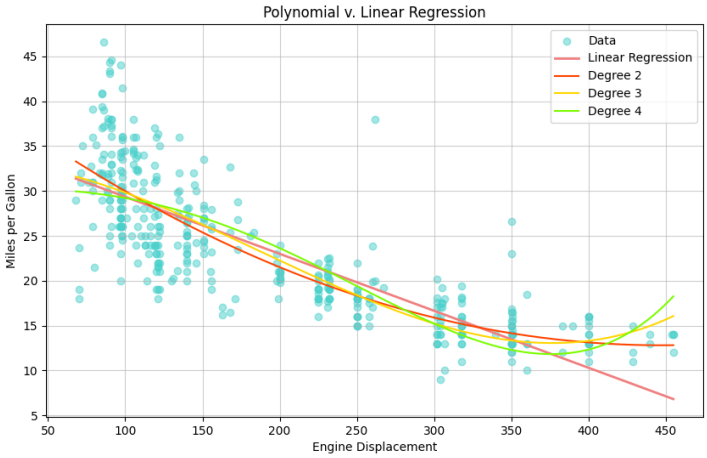
    plt.plot(x_plot, y_curve, color=color, label=f"Degree {degree}")


plt.xlabel("Engine Displacement")
plt.ylabel("Miles per Gallon")
plt.title("Polynomial v. Linear Regression")

plt.legend()
plt.grid(True, alpha=0.6)
plt.show()
```

Output

Degree	MSE	R2 Score
Linear	21.9867	0.6414
2	19.1884	0.6870
3	19.5162	0.6817
4	19.6208	0.6800



Result

Successfully implemented polynomial regression and compared it with linear regression on the auto MPG dataset.

Experiment 3 **Date: 04/08/2026**

Ridge and Lasso Regression

Aim

Implement Ridge and Lasso regression on the Diabetes dataset. Compare the performance of these regularized models with standard linear regression.
Tasks:

- Load and preprocess the dataset.
- Implement Ridge and Lasso regression.
- Tune hyperparameters using cross-validation.
- Compare performance metrics (MSE, R-squared) with standard linear regression.

Algorithm

1. **Load and Preprocess Data:** Load the Diabetes dataset, split it into training and testing sets, and standardize the features since regularized models are sensitive to feature scales.
2. **Implement Standard Linear Regression:** Fit a baseline ordinary least squares linear regression model on the scaled data.
3. **Implement Ridge and Lasso Regression:** Define Ridge (L2 penalty) and Lasso (L1 penalty) regression models.
4. **Tune Hyperparameters:** Use cross-validation techniques to find the optimal regularization strength (α) for both Ridge and Lasso.
5. **Evaluate and Compare Models:** Compare performance metrics (MSE and R^2) of the tuned regularized models against standard linear regression.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LassoCV, RidgeCV
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Datasets/diabetes.csv")
X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y, train_size
    =0.2, random_state=42)

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

linear = LinearRegression()
linear.fit(X_train, y_train)
y_pred_linear = linear.predict(X_test)

alphas = [0.01, 0.1, 1, 10, 100]

ridge = RidgeCV(alphas=alphas, cv=5)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

lasso = LassoCV(alphas=alphas, cv=4, random_state=42)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

print("Errors for Linear Regression")
print(f"MSE = {mean_squared_error(y_test, y_pred_linear):.4f} \nR2 = {
    r2_score(y_test, y_pred_linear):.4f}\n")

print("Errors for Ridge Regression")
print(f"Best alpha = {ridge.alpha_}")
print(f"MSE = {mean_squared_error(y_test, y_pred_ridge):.4f} \nR2 = {
    r2_score(y_test, y_pred_ridge):.4f}\n")

print("Errors for Lasso Regression")
print(f"Best alpha = {lasso.alpha_}")
print(f"MSE = {mean_squared_error(y_test, y_pred_lasso):.4f} \nR2 = {
    r2_score(y_test, y_pred_lasso):.4f}\n")
```

Output

```
Errors for Linear Regression
MSE = 3371.1695
R2 = 0.4351

Errors for Ridge Regression
Best alpha = 100.0
MSE = 3641.3145
R2 = 0.3899

Errors for Lasso Regression
Best alpha = 10.0
MSE = 3732.5945
R2 = 0.3746
```

Result

Successfully implemented ridge and lasso regressions on the diabetes dataset.

Experiment 4 **Date: 04/08/2026**

Logistic Regression

Aim

Implement a logistic regression model to predict the likelihood of a disease using the Pima Indians Diabetes dataset. Compare the performance with and without feature scaling. Tasks:

- Load and preprocess the Pima Indians Diabetes dataset.
- Implement logistic regression for binary classification.
- Evaluate model performance with and without feature scaling.
- Analyze metrics such as accuracy, precision, recall, and F1-score.

Algorithm

1. **Load and Preprocess Data:** Load the Diabetes dataset, split it into training and testing sets, and standardize the features since regularized models are sensitive to feature scales.
2. **Implement Standard Linear Regression:** Fit a baseline ordinary least squares linear regression model on the scaled data.
3. **Implement Ridge and Lasso Regression:** Define Ridge (L2 penalty) and Lasso (L1 penalty) regression models.
4. **Tune Hyperparameters:** Use cross-validation techniques to find the optimal regularization strength (α) for both Ridge and Lasso.
5. **Evaluate and Compare Models:** Compare performance metrics (accuracy, precision, recall, and f1 score) of the tuned regularized models against standard linear regression.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score
import pandas as pd

df = pd.read_csv("Datasets/pimadiabetes.csv")
df["class"] = df["class"].map({
    "tested_negative": 0,
    "tested_positive": 1
})
X = df.drop(columns=["class"])
```

```
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

model_no_scaling = LogisticRegression(max_iter=1000)
model_no_scaling.fit(X_train, y_train)
y_pred_no = model_no_scaling.predict(X_test)

print("Without feature scaling:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_no)} \nPrecision: {
   precision_score(y_test, y_pred_no)} \nRecall: {recall_score(y_test,
   y_pred_no)} \nF1 Score: {f1_score(y_test, y_pred_no)}")

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model_scaled = LogisticRegression(max_iter=1000)
model_scaled.fit(X_train_scaled, y_train)
y_pred_scaled = model_scaled.predict(X_test_scaled)

print("\nWith feature scaling:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_scaled)} \nPrecision: {
   precision_score(y_test, y_pred_scaled)} \nRecall: {recall_score(
   y_test, y_pred_scaled)} \nF1 Score: {f1_score(y_test, y_pred_scaled)}
   ")
```

Output

```
Scaled? Accuracy Precison Recall F1 Score
     No   0.7468   0.6379 0.6727   0.6549
    Yes   0.7532   0.6491 0.6727   0.6607
```

Result

Successfully implemented logistic regression on the Pima Indian diabetes dataset and compared the results with and without feature scaling.

Experiment 5 **Date: 13/08/2026**

Naive Bayes Classifier

Aim

Implement a Naïve Bayes classifier to categorize text documents into topics using the 20 Newsgroups dataset. Compare the performance of Multinomial Naïve Bayes with Bernoulli Naïve Bayes.
Tasks:

- Load and preprocess the 20 Newsgroups dataset.
- Implement Multinomial Naïve Bayes and Bernoulli Naïve Bayes classifiers.
- Evaluate and compare the performance of both models using metrics such as accuracy and F1-score.
- Discuss the strengths and weaknesses of each Naïve Bayes variant for text classification.

Algorithm

1. Load a subset of the 20 Newsgroups dataset and split into train and test sets.
2. Preprocess text documents through lowercasing, tokenization, and stop-word removal.
3. Vectorize text using term-frequency counts for the Multinomial variant.
4. Binarize text features (presence or absence) for the Bernoulli variant.
5. Train the Multinomial Naïve Bayes classifier on word-count distributions.
6. Train the Bernoulli Naïve Bayes classifier on binary feature indicators.
7. Evaluate both models using accuracy, precision, recall, and macro F1-scores.
8. Compare results to demonstrate how frequency modeling benefits text classification.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, f1_score
import pandas as pd

df = pd.read_csv("Datasets/20newsgroups.csv")
X = df["text"].fillna("")
y = df["category"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

vectorizer_multi = CountVectorizer(stop_words="english", max_features
   =5000)
X_train_multi = vectorizer_multi.fit_transform(X_train)
X_test_multi = vectorizer_multi.transform(X_test)

multi = MultinomialNB()
multi.fit(X_train_multi, y_train)
y_pred_multi = multi.predict(X_test_multi)

print("Multinomial Naive Bayes")
print(f"Accuracy: {accuracy_score(y_test, y_pred_multi)}")
print(f"F1 Score: {f1_score(y_test, y_pred_multi, average="weighted")}")

vectorizer_binary = CountVectorizer(stop_words="english", max_features
   =5000, binary=True)
X_train_binary = vectorizer_binary.fit_transform(X_train)
X_test_binary = vectorizer_binary.transform(X_test)

bernoulli = BernoulliNB()
bernoulli.fit(X_train_binary, y_train)
y_pred_binary = bernoulli.predict(X_test_binary)

print("\nBernoulli Naive Bayes")
print(f"Accuracy: {accuracy_score(y_test, y_pred_binary)}")
print(f"F1 Score: {f1_score(y_test, y_pred_binary, average="weighted")}"
   )
```

Output

```
Multinomial Naive Bayes
Accuracy:  0.6526
F1 Score:  0.6377

Bernoulli Naive Bayes
Accuracy:  0.5500
F1 Score:  0.5745
```

Result

Successfully implemented a Naive Bayes classifier on the 20 News Groups dataset and compared the results for binary and multinomial classifiers.

Experiment 6 **Date: 13/08/2026**

Logistic Regression With Regularization

Aim

Estimate the parameters of a logistic regression model using MLE and MAP on the Breast Cancer Wisconsin dataset. Compare the results and discuss the effects of regularization. Tasks:

- Load and preprocess the dataset.
- Implement logistic regression with MLE.
- Apply MAP estimation with different regularization priors (L1 and L2 regularization).
- Compare the performance and parameter estimates with MLE and MAP.

Algorithm

1. Load the Breast Cancer Wisconsin dataset and split it into training and testing subsets.
2. Standardize features to ensure fair penalty scaling during regularization.
3. Formulate Maximum Likelihood Estimation (MLE) using unpenalized binary cross-entropy loss.
4. Optimize MLE parameters via gradient ascent or iterative numerical solvers.
5. Formulate Maximum A Posteriori (MAP) estimation by adding parameter priors to the objective.
6. Implement L2 regularization (Gaussian prior) to penalize large parameter magnitudes.
7. Implement L1 regularization (Laplacian prior) to induce parameter sparsity and feature selection.
8. Compare test performance metrics and analyze how regularization prevents overfitting.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Datasets/breastcancer.csv")
X = df.drop(columns=["target"])
y = df[["target"]]
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size
    =0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MLE
mle = LogisticRegression(penalty=None, max_iter=1000)
mle.fit(X_train, y_train)
y_pred_mle = mle.predict(X_test)

# MAP Using L1
map_l1 = LogisticRegression(penalty="l1", solver='liblinear', C=1.0,
    max_iter=1000)
map_l1.fit(X_train, y_train)
y_pred_l1 = map_l1.predict(X_test)

# MAP using L2
map_l2 = LogisticRegression(penalty="l2", C=1.0, max_iter=1000)
map_l2.fit(X_train, y_train)
y_pred_l2 = map_l2.predict(X_test)

print("Logistic Regression MLE: ")
print(f"Accuracy: {accuracy_score(y_test, y_pred_mle): .4f}\nPrecision:
    {precision_score(y_test, y_pred_mle): .4f} \nRecall: {recall_score(
    y_test, y_pred_mle): .4f} \nF1 Score: {f1_score(y_test, y_pred_mle):
    .4f}\n")

print("Logistic Regression MAP (L1): ")
print(f"Accuracy: {accuracy_score(y_test, y_pred_l1): .4f}\nPrecision: {
    precision_score(y_test, y_pred_l1): .4f} \nRecall: {recall_score(
    y_test, y_pred_l1): .4f} \nF1 Score: {f1_score(y_test, y_pred_l1): .4
    f}\n")

print("Logistic Regression MAP (L2): ")
print(f"Accuracy: {accuracy_score(y_test, y_pred_l2): .4f}\nPrecision: {
    precision_score(y_test, y_pred_l2): .4f} \nRecall: {recall_score(
    y_test, y_pred_l2): .4f} \nF1 Score: {f1_score(y_test, y_pred_l2): .4
    f}\n")
```

Output

```
MLE
Accuracy:  0.9211
Precision:  0.9701
Recall:  0.9028
F1 Score:  0.9353

MAP L1
Accuracy:  0.9912
Precision:  0.9863
Recall:  1.0000
F1 Score:  0.9931

MAP L2
Accuracy:  0.9825
Precision:  0.9861
Recall:  0.9861
F1 Score:  0.9861
```

Result

Successfully estimated data with MLE, L1, and L2 priors with logistic regression.

Experiment 7 **Date: 13/08/2026**

MLE and MAP

Aim

Use MLE and MAP to estimate the parameters of a multinomial distribution on the 20Newsgroups dataset. Explore the impact of different priors on the estimation.
Tasks:

- Load and preprocess the dataset.
- Implement MLE for multinomial distribution parameter estimation.
- Apply MAP estimation with various priors (e.g., Dirichlet priors).
- Compare results and evaluate the effect of different priors.

Algorithm

1. Load a subset of the 20 Newsgroups dataset and extract text documents.
2. Tokenize documents and construct a shared vocabulary index.
3. Compute term frequency counts across the text corpus.
4. Implement Maximum Likelihood Estimation (MLE) using raw relative word frequencies.
5. Handle zero-probability issues caused by unseen words in MLE parameter estimation.
6. Formulate Maximum A Posteriori (MAP) estimation using a conjugate Dirichlet prior.
7. Apply Laplace smoothing (uniform Dirichlet prior) to regularize parameter estimates.
8. Experiment with informative (non-uniform) Dirichlet priors to bias parameter weights.
9. Compare estimated parameter distributions and evaluate the impact of different prior configurations.

Program

```
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = pd.read_csv("Datasets/20newsgroups.csv")
```

```
texts = df["text"].fillna("")

vectorizer = CountVectorizer(stop_words="english", max_features=1000)
X = vectorizer.fit_transform(texts)
vocabulary = vectorizer.get_feature_names_out()
word_counts = np.asarray(X.sum(axis=0)).flatten()

N = word_counts.sum()
V = len(vocabulary)

print(f"Vocabulary size: {V}")
print(f"Total words: {N}")

mle = word_counts / N

alphas = [1, 2, 5, 10]
map_results = {}
for alpha in alphas:
    # Symmetric Dirichlet(alpha) prior
    map_estimate = (word_counts + alpha - 1) / (N + V * (alpha - 1))
    map_results[alpha] = map_estimate


compare_words = ["computer", "windows", "god", "space", "game"]

print("\nComparison of word probabilities")
print("-" * 70)

header = f"{'Word':<15}{'MLE':<12}"
for alpha in alphas:
    header += f"MAP a={alpha:<8}"
print(header)

for word in compare_words:
    if word in vocabulary:
        idx = np.where(vocabulary == word)[0][0]
        row = f"{word:<15}{mle[idx]:<12.6f}"
        for alpha in alphas:
            row += f"{map_results[alpha][idx]:<12.6f}"
        print(row)


def print_top_words(probabilities, title):
    top = np.argsort(probabilities)[-10:][::-1]
    print(f"\n{title}")
    print("-" * 40)
    for i in top:
        print(f"{vocabulary[i]:<15} {probabilities[i]:.6f}")


print_top_words(mle, "Top 10 Words - MLE")

for alpha in alphas:
    print_top_words(
        map_results[alpha],
        f"Top 10 Words - MAP (a={alpha})"
    )
```

Output

```
Vocabulary size: 1000
Total words: 840873

Comparison of word probabilities
------------------------------------------------------------------------
Word            MLE         MAP α=1       MAP α=2       MAP α=5       MAP α=10
computer        0.001558    0.001558    0.001557    0.001555    0.001552
windows         0.002727    0.002727    0.002725    0.002719    0.002709
god             0.004042    0.004042    0.004039    0.004028    0.004010
space           0.002188    0.002188    0.002187    0.002183    0.002176
game            0.001792    0.001792    0.001791    0.001788    0.001784

Top 10 Words - MLE
--------------------------------------
ax              0.074204
like            0.007760
don             0.007759
people          0.007680
just            0.007340
know            0.006854
use             0.005978
think           0.005947
time            0.005788
max             0.005515
```

```
Top 10 Words - MAP (α=1)
--------------------------------------
ax              0.074204
like            0.007760
don             0.007759
people          0.007680
just            0.007340
know            0.006854
use             0.005978
think           0.005947
time            0.005788
max             0.005515

Top 10 Words - MAP (α=2)
--------------------------------------
ax              0.074117
like            0.007752
don             0.007751
people          0.007672
just            0.007332
know            0.006847
use             0.005972
think           0.005942
time            0.005782
max             0.005509
```

```
Top 10 Words - MAP (α=5)
--------------------------------------
ax              0.073857
like            0.007728
don             0.007727
people          0.007648
just            0.007310
know            0.006826
use             0.005955
think           0.005924
time            0.005765
max             0.005493

Top 10 Words - MAP (α=10)
--------------------------------------
ax              0.073429
like            0.007688
don             0.007687
people          0.007609
just            0.007273
know            0.006792
use             0.005926
think           0.005895
time            0.005737
max             0.005467
```

Result

Successfully implemented MLE and MAP on the 20 News Groups dataset and compared effects of different priors.

Experiment 8 **Date: 01/09/2026**

K Nearest Neighbors

Aim

Implement the K-Nearest Neighbors (KNN) algorithm for image classification using the Fashion MNIST dataset. Experiment with different values of K and analyze their impact on model performance.
Tasks:

- Load and preprocess the Fashion MNIST dataset.
- Implement KNN for multi-class classification.
- Experiment with different values of K and evaluate performance.
- Discuss the impact of different K values on model accuracy and computational efficiency.

Algorithm

1. Load the Fashion MNIST dataset containing 28x28 grayscale clothing images.
2. Normalize pixel intensity values to a standard range (e.g., $[0, 1]$).
3. Flatten 2D image matrices into 1D feature vectors for distance computation.
4. Sample a manageable subset of the dataset to mitigate KNN's high computational complexity during inference.
5. Split the preprocessed data into training and testing sets.
6. Implement the K-Nearest Neighbors classifier using distance metrics like Euclidean distance.
7. Train models across a range of different K values (e.g., $1, 3, 5, 11$).
8. Evaluate classification performance using accuracy, precision, and recall metrics for each K.
9. Analyze how small K leads to overfitting (high variance) and large K leads to underfitting (high bias).
10. Discuss the computational trade-offs and slow inference times associated with lazy learning algorithms.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score
import pandas as pd
import time

df = pd.read_csv("Datasets/fashion_mnist.csv")
df = df.sample(10000, random_state=42) # take only 10,000 samples
X = df.drop(columns=["label"])
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

X_train = X_train / 255.0
X_test = X_test / 255.0

k_vals = [1, 3, 5, 7, 9]
for k in k_vals:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    t0 = time.perf_counter()
    y_pred = model.predict(X_test)
    elapsed = time.perf_counter() - t0

    acc = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="macro",
   zero_division=0)
    recall = recall_score(y_test, y_pred, average="macro", zero_division
   =0)
    f1 = f1_score(y_test, y_pred, average="macro", zero_division=0)

    print(f"K = {k}")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall   : {recall:.4f}")
    print(f"F1 Score : {f1:.4f}")
    print(f"Time     : {elapsed:.4f}s\n")
```

Output

K Value	Accuracy	Precision	Recall	F1 Score	Time (s)
2	0.7980	0.8038	0.7980	0.7942	0.13
4	0.8150	0.8206	0.8150	0.8135	0.04
6	0.8140	0.8204	0.8140	0.8133	0.03
8	0.8110	0.8174	0.8110	0.8107	0.03
10	0.8090	0.8155	0.8090	0.8084	0.04

Result

Successfully implemented KNN on the Fashion MNIST dataset and compared different k-values.

Experiment 9 **Date: 01/09/2026**

Decision Tree using ID3

Aim

Implement a Decision Tree classifier using the ID3 algorithm to segment customers based on their purchasing behavior using the Iris dataset. Analyze the tree structure and discuss the feature importance.
Tasks:

- Load and preprocess the Iris dataset.
- Implement Decision Tree using the ID3 algorithm.
- Visualize the decision tree and analyze feature importance.
- Discuss how the tree structure helps in understanding customer behavior.

Algorithm

1. Load the preprocessed dataset and ensure all feature attributes are discretized into categorical values.
2. Compute the total entropy of the dataset based on the class distribution of the target variable.
3. For each available feature, calculate the conditional entropy after splitting the dataset on that feature's attributes.
4. Compute the Information Gain for each feature by subtracting its conditional entropy from the total dataset entropy.
5. Select the feature that yields the maximum Information Gain to be the current decision node.
6. Partition the dataset into subsets corresponding to the discrete branches of the selected feature.
7. Recursively repeat the entropy and information gain calculations for each subset to construct child nodes.
8. Terminate recursion when all instances in a node belong to a single target class or when no remaining features are left.
9. Output the fully constructed decision tree structure to analyze split paths and evaluate feature importance.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(r'Datasets/iris.csv')

X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

model = DecisionTreeClassifier(criterion="entropy", max_depth=3,
   random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Accuracy = {accuracy_score(y_test, y_pred):.4f}\n")

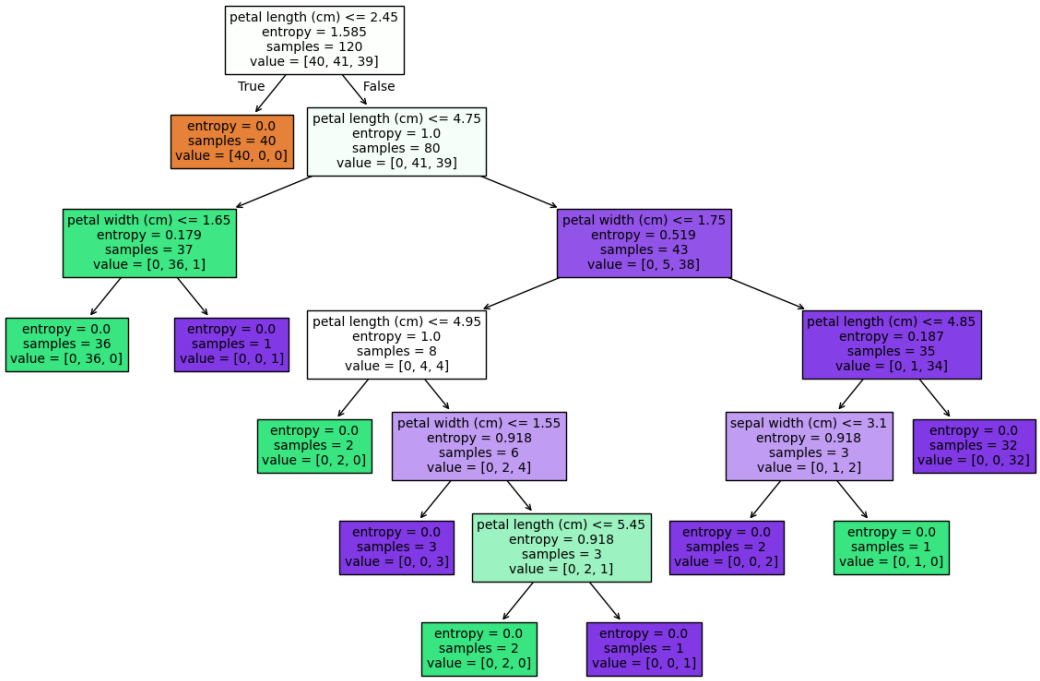
for feature, importance in zip(X.columns, model.feature_importances_):
  print(f"{feature}: {importance: .4f}")

plt.figure(figsize=(20, 10))
plot_tree(model, feature_names=X.columns, filled=True, fontsize=10)
plt.show()
```

Output

```
Accuracy = 1.0000

Feature importances:
sepal length (cm):  0.0000
sepal width (cm):  0.0145
petal length (cm):  0.8954
petal width (cm):  0.0901
```



Result

Successfully implemented the ID3 algorithm on the Iris dataset.

Experiment 10 **Date: 08/09/2026**

Support Vector Machines

Aim

Implement a Linear Support Vector Machine (SVM) to classify the Iris dataset. Visualize the decision boundary and discuss how the margin is determined.
Tasks:

- Load and preprocess the Iris dataset.
- Implement a Linear SVM for binary classification (e.g., classify Setosa vs. Non-Setosa).
- Visualize the decision boundary and margin. Discuss the concept of the margin and how it influences classification.

Algorithm

1. Load the Iris dataset, select two features for 2D visualization, and filter for binary classification.
2. Standardize the feature values and split the data into training and testing sets.
3. Formulate the linear SVM optimization problem to maximize the margin subject to classification constraints.
4. Train the linear SVM model to find the optimal hyperplane and identify the support vectors.
5. Visualize the decision boundary, the margin boundaries, and the support vectors alongside the data points.
6. Analyze how the margin width and the regularization parameter influence the model's performance.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

df = pd.read_csv("Datasets/iris.csv")
X = df[["petal length (cm)", "petal width (cm)"]].values
y = df["target"].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
    =42, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel="linear", C=1.0)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print(f"\nAccuracy = {accuracy_score(y_test, pred): .4f}")

plt.figure(figsize=(8, 6))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="coolwarm",
    edgecolors='k')

w = model.coef_[0]
b = model.intercept_[0]

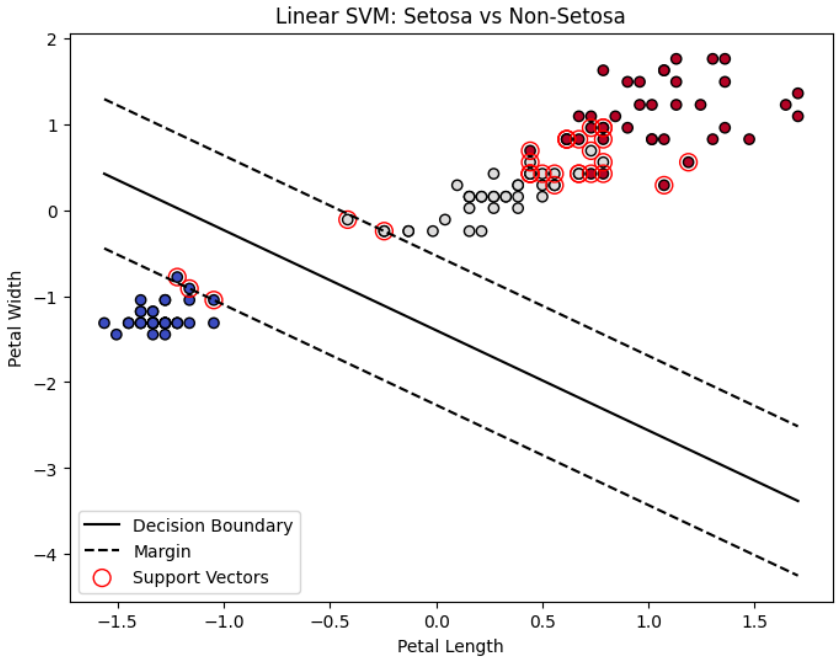
x = np.linspace(X_train[:, 0].min(), X_train[:, 0].max(), 100)
y_boundary = -(w[0] * x + b) / w[1]
y_margin1 = -(w[0] * x + b - 1) / w[1]
y_margin2 = -(w[0] * x + b + 1) / w[1]

plt.plot(x, y_boundary, 'k-', label='Decision Boundary')
plt.plot(x, y_margin1, 'k--', label='Margin')
plt.plot(x, y_margin2, 'k--')

plt.scatter(
    model.support_vectors_[:, 0],
    model.support_vectors_[:, 1],
    s=100,
    facecolors='none',
    edgecolors='red',
    label='Support Vectors'
)

plt.xlabel("Petal Length (scaled)")
plt.ylabel("Petal Width (scaled)")
plt.title("Linear SVM: Iris Classification")
plt.legend()
plt.savefig("Datasets/iris_svm.png", dpi=150, bbox_inches='tight')
plt.show()
```

Output



Result

Successfully classified data into setosa and non-setosa classes using a linear SVM on the Iris dataset.

Experiment 11 **Date: 15/09/2026**

SVM Kernels

Aim

Implement and compare the performance of SVM classifiers with linear, polynomial, and RBF kernels on the Fashion MNIST dataset. Analyze the advantages and disadvantages of each kernel type.
Tasks:

- Load and preprocess the Fashion MNIST dataset.
- Implement SVM with linear, polynomial, and RBF kernels.
- Compare the classification performance for each kernel.
- Discuss the strengths and weaknesses of each kernel type.

Algorithm

1. Load the Fashion MNIST dataset and normalize pixel intensity values to the range $[0, 1]$.
2. Flatten the 2D image matrices into 1D feature vectors and split into training and testing sets.
3. Implement a Support Vector Machine using a linear kernel to construct flat decision boundaries.
4. Implement an SVM with a polynomial kernel to capture non-linear feature interactions.
5. Implement an SVM with a Radial Basis Function (RBF) kernel to map inputs into infinite-dimensional space.
6. Train all three kernel variants and evaluate their classification performance using accuracy, precision, recall, and F1-score metrics.
7. Analyze strengths and weaknesses: note that linear kernels are fast but underfit complex shapes, polynomial kernels risk high computational costs and overfitting, and RBF kernels handle intricate boundaries effectively at the expense of heavy memory and training time.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, recall_score,
   precision_score, f1_score
import pandas as pd

df = pd.read_csv("Datasets/fashion_mnist.csv")
df = df.sample(5000, random_state=42) # take 5000 samples (optimization)
X = df.drop(columns=["label"])
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

kernels = ["linear", "poly", "rbf"]
acs = [] ; pcs = [] ; rcs = []; f1s = []
for k in kernels:
  if k == 'poly':
    model = SVC(kernel="poly", degree=3, max_iter=1000, random_state=42)
  else:
    model = SVC(kernel=k, max_iter=1000, random_state=42)

  model.fit(X_train, y_train)
  pred = model.predict(X_test)

  acs.append(accuracy_score(y_test, pred))
  pcs.append(precision_score(y_test, pred, average='weighted'))
  rcs.append(recall_score(y_test, pred, average='weighted'))
  f1s.append(f1_score(y_test, pred, average='weighted'))
results = pd.DataFrame({
    "Kernel": kernels,
    "Accuracy":acs,
    "Precision":pcs,
    "Recall":rcs,
    "F1 Score":f1s
})
numeric = ["Accuracy", "Precision", "Recall", "F1 Score"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))
```

Output

Kernel	Accuracy	Precision	Recall	F1 Score
linear	0.7685	0.7693	0.7685	0.7679
poly	0.8450	0.8595	0.8450	0.8494
rbf	0.8660	0.8646	0.8660	0.8644

Result

Successfully implemented support vector machines with linear, polynomial (degree=3), and RBF kernels on the Fashion MNIST dataset.

Experiment 12 **Date: 15/09/2026**

Multilayer Feed-Forward Networks

Aim

Implement and train a Multilayer Feed-Forward Network (MLP) on the Wine Quality dataset. Experiment with different numbers of hidden layers and neurons, and discuss how these choices affect the network's performance.
Tasks:

- Load and preprocess the Wine Quality dataset.
- Design and implement an MLP with varying architectures (different hidden layers and neurons).
- Train and evaluate the network.
- Discuss the impact of architecture choices on performance.

Algorithm

1. Load the Wine Quality dataset, handle missing values, and split into training and testing sets.
2. Standardize input features to ensure stable gradient descent and faster convergence during backpropagation.
3. Design a baseline Multilayer Feed-Forward Network (MLP) featuring input, hidden, and output layers.
4. Configure appropriate loss functions, activation functions (e.g., ReLU), and optimizers (e.g., Adam).
5. Train multiple network variants with varying depths (different numbers of hidden layers) and widths (different neuron counts).
6. Evaluate performance using relevant metrics (e.g., accuracy, F1-score, or mean squared error).
7. Discuss how architectural choices affect model capacity, risk of overfitting, and generalization capabilities.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score
```

```
import pandas as pd

df = pd.read_csv("Datasets/wine_quality.csv")
X = df.drop(columns=["class"])
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

archs = [(10,), (100,), (20,10), (50, 100, 30)]
accs = [] ; precs = [] ; recs = [] ; f1s = []
for arch in archs:
  model = MLPClassifier(hidden_layer_sizes=arch, max_iter=2000,
   early_stopping=True, random_state=42)
  model.fit(X_train, y_train)
  pred = model.predict(X_test)

  accs.append(accuracy_score(y_test, pred))
  precs.append(precision_score(y_test, pred, average='weighted',
   zero_division=0))
  recs.append(recall_score(y_test, pred, average='weighted'))
  f1s.append(f1_score(y_test, pred, average='weighted'))
results = pd.DataFrame({
    "Architecture": [str(a) for a in archs],
    "Accuracy": accs,
    "Precision": precs,
    "Recall": recs,
    "F1 Score": f1s,
})
numeric = ["Accuracy", "Precision", "Recall", "F1 Score"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))
```

Output

Architecture	Accuracy	Precision	Recall	F1 Score
(10,)	0.5278	0.7204	0.5278	0.4479
(100,)	0.7222	0.7960	0.7222	0.7152
(20, 10)	0.8889	0.9046	0.8889	0.8885
(50, 100, 30)	0.8889	0.9206	0.8889	0.8889

Result

Successfully implemented multilayer feed-forward networks on the Wine Quality dataset and compared the performance of different architectures.

Experiment 13 **Date: 15/09/2026**

Neural Networks

Aim

Implement and compare the performance of a neural network using different activation functions (Sigmoid, ReLU, Tanh) on the MNIST dataset. Analyze how each activation function affects the training process and classification accuracy.
Tasks:

- Load and preprocess the MNIST dataset.
- Implement neural networks using Sigmoid, ReLU, and Tanh activation functions.
- Train and evaluate each network.
- Compare training times, convergence, and classification accuracy.

Algorithm

1. Load the MNIST dataset and scale pixel intensity values to the range $[0, 1]$.
2. Flatten the 2D image matrices into 1D feature vectors and split into training and testing sets.
3. Build a baseline neural network architecture with configurable hidden layers and a softmax output layer.
4. Implement the Sigmoid activation function to map hidden layer outputs to the range $(0, 1)$.
5. Implement the Tanh activation function to map hidden layer outputs to a zero-centered range $(-1, 1)$.
6. Implement the ReLU (Rectified Linear Unit) activation function to output positive values directly while thresholding negatives.
7. Train each network variant independently using identical hyperparameter configurations and optimizers.
8. Compare training times, convergence rates, and test classification accuracies.
9. Analyze how Sigmoid and Tanh suffer from vanishing gradients while ReLU enables faster and deeper convergence.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score,
   recall_score, f1_score
import matplotlib.pyplot as plt
import pandas as pd
import time

df = pd.read_csv("Datasets/mnist.csv")
df = df.sample(5000, random_state=42)
X = df.drop(columns=["label"])
y = df["label"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

activations = ["logistic", "relu", "tanh"]
acs = [] ; pcs = [] ; rcs = [] ; f1s = []; ts = [] ; models = [] ; iters
    = []
for a in activations:
  model = MLPClassifier(hidden_layer_sizes=(50,), activation=a, max_iter
   =1000, early_stopping=True, random_state=42)
  start = time.time()
  model.fit(X_train, y_train)
  ts.append(time.time() - start)
  iters.append(model.n_iter_)

  pred = model.predict(X_test)

  models.append(model)
  acs.append(accuracy_score(y_test, pred))
  pcs.append(precision_score(y_test, pred, average='weighted'))
  rcs.append(recall_score(y_test, pred, average='weighted'))
  f1s.append(f1_score(y_test, pred, average='weighted'))
results = pd.DataFrame({
    "activation": activations,
    "accuracy":acs,
    "precision":pcs,
    "recall":rcs,
    "f1 score":f1s,
    "iterations":iters,
    "time": ts
})
numeric = ["accuracy", "precision", "recall", "f1 score", "time"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))
```

Output

activation	accuracy	precision	recall	f1 score	iterations	time
Sigmoid	0.9060	0.9057	0.9060	0.9055	42	12.8744
ReLU	0.9170	0.9168	0.9170	0.9167	29	3.7127
Tanh	0.9070	0.9069	0.9070	0.9064	41	8.2921

Result

Successfully implemented and compared the performances of neural networks with Sigmoid, ReLU, and Tanh activation functions on the MNIST dataset.

Experiment 14 **Date: 24/09/2026**

Hierarchical and Partitional Clustering

Aim

Implement and compare hierarchical (agglomerative) and partitional (K-means) clustering algorithms on the Mall Customers dataset. Discuss the strengths and weaknesses of each method based on clustering results and evaluation metrics.
Tasks:

- Load and preprocess the Mall Customers dataset.
- Apply both hierarchical (agglomerative) and K-means clustering.
- Compare results using metrics such as inertia, silhouette score, and clustering visualization.
- Discuss the advantages and disadvantages of each clustering method.

Algorithm

1. Load the Mall Customers dataset and select relevant features for customer segmentation.
2. Standardize feature values to ensure equal weighting during distance calculations.
3. Implement K-means clustering, determining the optimal number of clusters (k) via the Elbow method.
4. Implement Agglomerative Hierarchical clustering using linkage criteria.
5. Evaluate both models using Silhouette scores and cluster inertia.
6. Visualize the resulting clusters using 2D scatter plots.
7. Compare strengths and weaknesses: note K-means' computational efficiency versus hierarchical clustering's intuitive tree-based structures and lack of a pre-defined k requirement.

Program

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("Datasets/Mall_Customers.csv")
```

```
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=5, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

print("\nK Means")
print(f"Inertia = {kmeans.inertia_: .4f} \nSillhouette Score = {
    silhouette_score(X_scaled, kmeans_labels): .4f}")

agg = AgglomerativeClustering(n_clusters=5)
agg_labels = agg.fit_predict(X_scaled)

print("\nAgglomerative Clustering")
print(f"Sillhouette Score = {silhouette_score(X_scaled, agg_labels): .4f
    }")

plt.figure(figsize=(12, 5))
plt.subplot(1,2,1)
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=kmeans_labels, alpha=0.5,cmap="
    inferno")

plt.title("K Means")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")

plt.subplot(1,2,2)
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=agg_labels, alpha=0.5, cmap="
    magma")

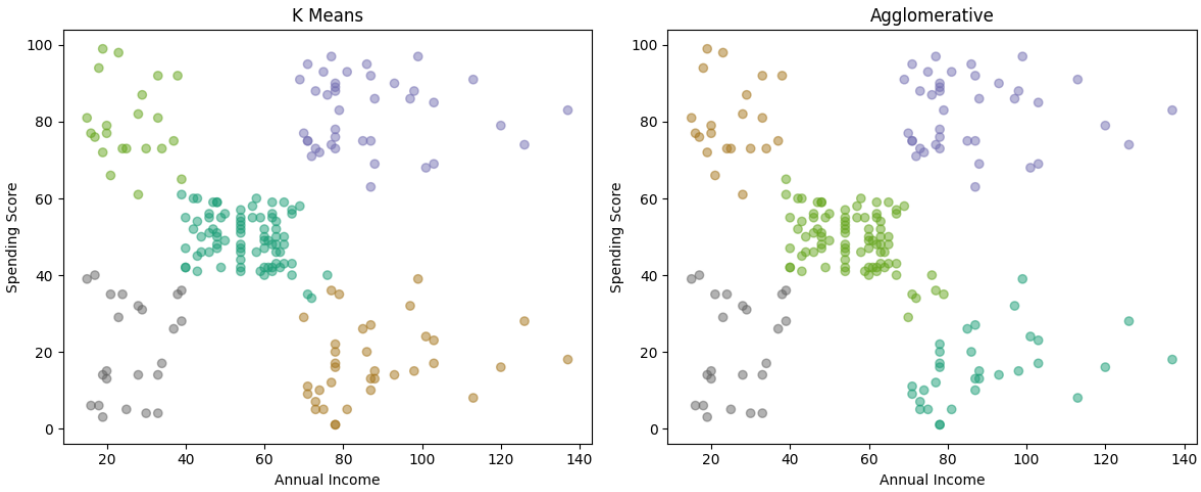
plt.title("Agglomerative (Hierarchical)")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")

plt.tight_layout()
plt.show()
```

Output

```
K Means (n=5)
Inertia = 65.5684
Silhouette Score = 0.5547

Agglomerative (n=5)
Silhouette Score = 0.5538
```



Result

Successfully implemented partitional (K Means) and hierarchical (agglomerative) clustering on the Mall Customers dataset.

Experiment 15 **Date: 24/09/2026**

K-Means Clustering

Aim

Implement and apply K-means clustering to the Digits dataset. Experiment with different numbers of clusters and evaluate the clustering results using metrics such as inertia and silhouette score. Analyze how the choice of K affects clustering performance.
Tasks:

- Load and preprocess the Digits dataset.
- Implement K-means clustering with various numbers of clusters.
- Evaluate clustering performance using inertia and silhouette score.
- Analyze the impact of the number of clusters on clustering quality.

Algorithm

1. Load the Digits dataset and normalize feature values to scale pixel intensities consistently.
2. Implement K-means clustering across a range of different cluster counts (K).
3. Compute cluster inertia (within-cluster sum of squares) for each K to apply the Elbow method.
4. Calculate Silhouette scores for each K to measure cluster cohesion and separation quality.
5. Evaluate the alignment between unsupervised clusters and true digit labels using external validation metrics if applicable.
6. Analyze how the choice of K impacts clustering performance and trade-offs between granularity and grouping error.

Program

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA # for visualization
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = pd.read_csv("Datasets/digits.csv")
X = df.drop(columns=["target"])
X_scaled = StandardScaler().fit_transform(X)
```

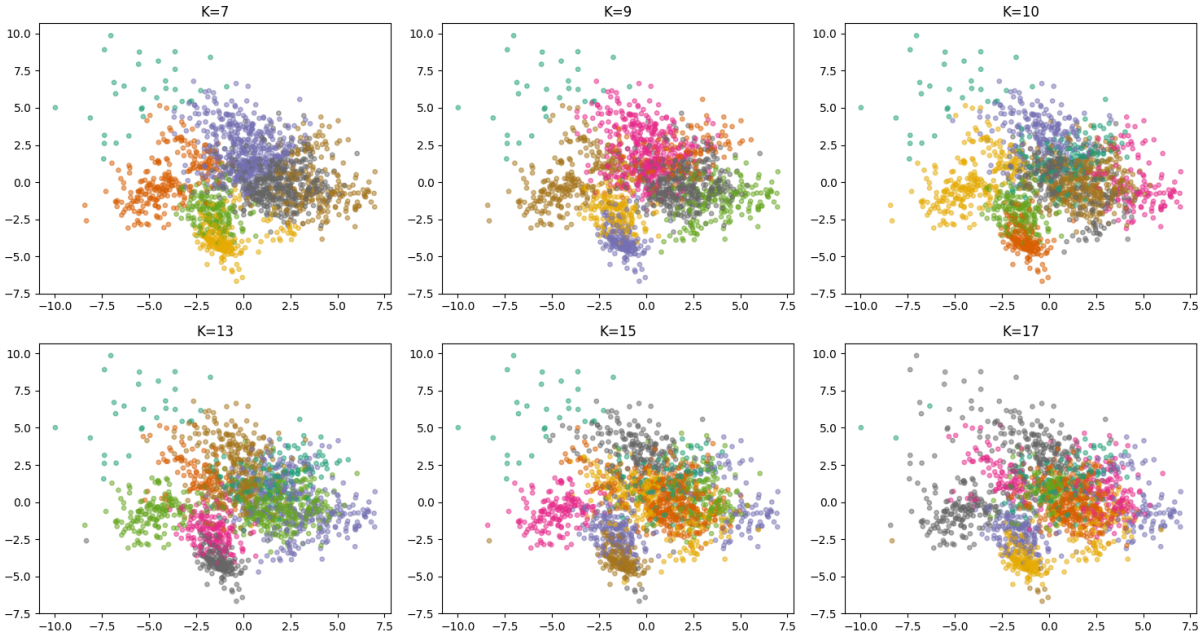
```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_scaled)
ks = [4, 5, 7, 9, 11, 13]
silhs = [] ; labs = [] ; ins = []
for k in ks:
  model = KMeans(n_clusters=k, random_state=42, n_init=10)
  labels = model.fit_predict(X_scaled)
  labs.append(labels)
  ins.append(model.inertia_)
  silhs.append(silhouette_score(X_scaled, labels))

results = pd.DataFrame({
    "K_Value":ks,
    "Inertia":ins,
    "Silhouette_Score":silhs
})
numeric = ["Inertia", "Silhouette_Score"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(15,8))
axes = axes.flatten()
for i in range(6):
  axes[i].scatter(X_reduced[:,0], X_reduced[:,1], c=labs[i], cmap="
   inferno", alpha=0.5, s=15)
  axes[i].set_title(f"K={ks[i]}")
plt.tight_layout()
plt.show()
```

Output

K_Value	Inertia	Silhouette_Score
7	72373.0996	0.1238
9	67117.1395	0.1415
10	65357.0476	0.1447
13	58469.8298	0.1540
15	55619.0657	0.1573
17	51957.1165	0.1643



Result

Successfully implemented K-means clustering on the digits dataset and compared performances for different values of K.

Experiment 16 **Date: 24/09/2026**

Bootstrapping and Cross-Validation

Aim

Implement bootstrapping and cross-validation on the Iris dataset. Compare the model performance metrics (e.g., accuracy, F1-score) obtained using these resampling methods. Discuss the advantages and disadvantages of each method.
Tasks:

- Load and preprocess the Iris dataset.
- Implement bootstrapping to generate multiple samples and evaluate the model.
- Implement k-fold cross-validation and evaluate the model.
- Compare the performance metrics and discuss the pros and cons of each resampling method.

Algorithm

1. Load and preprocess the Iris dataset to prepare feature matrices and target class labels.
2. Initialize a base classification model (e.g., K-Nearest Neighbors or Logistic Regression).
3. Implement bootstrapping by repeatedly sampling the dataset with replacement to create training sets.
4. Evaluate the model on out-of-bag (OOB) samples or held-out validation subsets for each bootstrap iteration.
5. Implement k-fold cross-validation by partitioning the dataset into k mutually exclusive folds.
6. Train and test the model iteratively across all folds, ensuring each subset serves as the validation set once.
7. Compute and record performance metrics (e.g., accuracy, F1-score) for both resampling methods.
8. Compare results and discuss trade-offs: bootstrapping utilizes sampling with replacement to handle small datasets but can introduce optimism bias, whereas k-fold cross-validation provides more reliable, unbiased error estimation.

Program

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, cross_val_score,
   KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.utils import resample
from sklearn.metrics import accuracy_score, f1_score

df = pd.read_csv("Datasets/iris.csv")

X = df.drop(columns=["target"])
y = df["target"]

scaler = StandardScaler()
X = scaler.fit_transform(X)

boot_accuracy = []
boot_f1 = []

for i in range(10):
    X_boot, y_boot = resample(X, y, replace=True, random_state=i)

    X_train, X_test, y_train, y_test = train_test_split(X_boot, y_boot,
   test_size=0.2, random_state=42)

    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    boot_accuracy.append(accuracy_score(y_test, y_pred))
    boot_f1.append(f1_score(y_test, y_pred, average="weighted"))

print("\nBootstrapping")
print(f"Average accuracy = {np.mean(boot_accuracy): .4f}")
print(f"Average f1 score = {np.mean(boot_f1): .4f}")

model = LogisticRegression(max_iter=1000)

kfold = KFold(n_splits=5, shuffle=True, random_state=42)

accuracy = cross_val_score(model, X, y, cv=kfold, scoring="accuracy")
f1 = cross_val_score(model, X, y, cv=kfold, scoring="f1_weighted")

print("\nK-Fold Cross Validation")
print(f"Average accuracy = {accuracy.mean(): .4f}")
print(f"Average f1 score = {f1.mean(): .4f}")
```

Output

```
        Method Mean Accuracy Mean F1
Bootstrapping        0.9533  0.9534
    K-Fold CV        0.9600  0.9599
```

Result

Successfully implemented bootstrapping and cross-validation (5-fold) on the Iris dataset.

Experiment 17 **Date: 24/09/2026**

Bagging and Boosting

Aim

Implement bagging and boosting ensemble methods on the Titanic dataset. Compare the performance of both methods in terms of accuracy, precision, recall, and F1-score. Discuss how each method improves model performance and their respective strengths and weaknesses.
Tasks:

- Load and preprocess the Titanic dataset.
- Implement bagging using a base classifier (e.g., decision tree) and evaluate performance.
- Implement boosting using a boosting algorithm (e.g., AdaBoost) and evaluate performance.
- Compare performance metrics and discuss the strengths and weaknesses of each method.

Algorithm

1. Load the Titanic dataset, handle missing values, encode categorical features, and split into training and testing subsets.
2. Implement a bagging ensemble (e.g., Bagged Decision Trees or Random Forest) using bootstrap sampling with replacement to train parallel base learners.
3. Implement a boosting ensemble (e.g., AdaBoost or Gradient Boosting) to train sequential base learners that focus on previously misclassified instances.
4. Train both ensemble models on the training data.
5. Evaluate model performance using accuracy, precision, recall, and F1-score metrics on the test set.
6. Compare results and discuss how bagging primarily reduces variance while boosting reduces bias, noting their trade-offs regarding computational cost and noise sensitivity.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
import pandas as pd

df = pd.read_csv("Datasets/titanic.csv")

# Only useful columns
df = df[["pclass","sex","age","sibsp","parch","fare","embarked","
    survived"]]

# Filing in null values
num_cols = ["age", "fare"]
num_imputer = SimpleImputer(strategy="median")
df[num_cols] = num_imputer.fit_transform(df[num_cols])

cat_cols = ["sex", "embarked"]
cat_imputer = SimpleImputer(strategy="most_frequent")
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

encoder = LabelEncoder()
df["sex"] = encoder.fit_transform(df["sex"])
df["embarked"] = encoder.fit_transform(df["embarked"])

X = df.drop(columns=["survived"])
y = df["survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
     random_state=42)

acs, pcs, rcs, f1s = [], [], [], []
models = {
    "Bagging": BaggingClassifier(estimator=DecisionTreeClassifier(),
    n_estimators=100, random_state=42),
    "Boosting": AdaBoostClassifier(n_estimators=100, random_state=42)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    acs.append(accuracy_score(y_test, pred))
    pcs.append(precision_score(y_test, pred))
    rcs.append(recall_score(y_test, pred))
    f1s.append(f1_score(y_test, pred))
results = pd.DataFrame({
    "Method": list(models.keys()),
    "Accuracy": acs,
    "Precision": pcs,
    "Recall": rcs,
    "F1 Score": f1s
})
numeric = ["Accuracy", "Precision", "Recall", "F1 Score"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))
```

Output

Method	Accuracy	Precision	Recall	F1 Score
Bagging	0.8053	0.8190	0.7288	0.7713
Boosting	0.7672	0.8000	0.6441	0.7136

Result

Successfully implemented bagging and boosting on the Titanic dataset.

Experiment 18 **Date: 24/09/2026**

Bias-Variance Tradeoff

Aim

Investigate the bias-variance tradeoff using polynomial regression on the Boston Housing dataset. Plot the training and validation errors for various polynomial degrees and discuss the tradeoff between bias and variance.
Tasks:

- Load and preprocess the Boston Housing dataset.
- Implement polynomial regression with varying degrees.Plot training and validation errors for each degree.
- Discuss the bias-variance tradeoff and its impact on model performance.

Algorithm

1. Load the Boston Housing dataset, handle missing values, and split into training and validation subsets.
2. Standardize input features to prevent numerical instability with higher-order polynomial terms.
3. Generate polynomial feature transformations across a range of increasing degrees (e.g., $d = 1$ to $d = 10$).
4. Fit polynomial regression models on the training set for each respective degree.
5. Compute training and validation errors (e.g., Mean Squared Error) for each model configuration.
6. Plot training and validation errors as a function of model complexity (polynomial degree).
7. Analyze the bias-variance tradeoff: low degrees exhibit high bias and underfitting, high degrees exhibit high variance and overfitting, and an optimal intermediate degree minimizes generalization error.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, root_mean_squared_error
from sklearn.datasets import fetch_openml
import matplotlib.pyplot as plt
import pandas as pd
```

```
boston = fetch_openml(name="boston", version=1, as_frame=True)
df = boston.frame
X = df.drop(columns=["MEDV"])
y = df["MEDV"]

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
    random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
   test_size=0.5, random_state=42)

degrees = [1, 2, 3, 4, 5]
tr_err, val_err = [], []
rmse_tr_err, rmse_val_err = [], []
for deg in degrees:
    poly = PolynomialFeatures(degree=deg)
    X_train_p = poly.fit_transform(X_train)
    X_val_p = poly.transform(X_val)

    scaler = StandardScaler()
    X_train_p = scaler.fit_transform(X_train_p)
    X_val_p = scaler.transform(X_val_p)

    model = LinearRegression()
    model.fit(X_train_p, y_train)
    train_pred = model.predict(X_train_p)
    val_pred = model.predict(X_val_p)

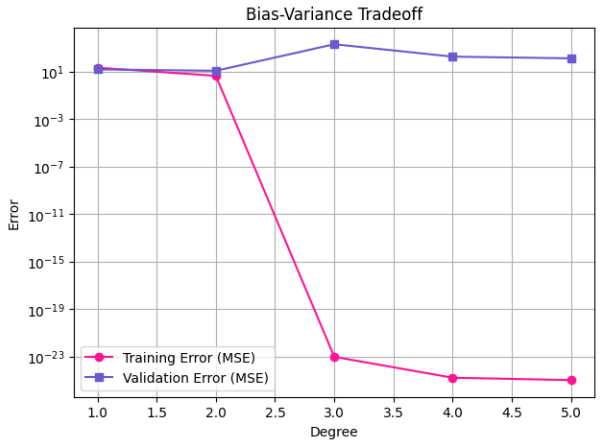
    tr_err.append(mean_squared_error(y_train, train_pred))
    val_err.append(mean_squared_error(y_val, val_pred))
    rmse_tr_err.append(root_mean_squared_error(y_train, train_pred))
    rmse_val_err.append(root_mean_squared_error(y_val, val_pred))
results = pd.DataFrame({
    "Degree": degrees,
    "Training Error": tr_err,
    "Validation Error": val_err,
    "Training Error (RMSE)": rmse_tr_err,
    "Validation Error (RMSE)": rmse_val_err
})
numeric = ["Training Error", "Validation Error", "Training Error (RMSE)"
   , "Validation Error (RMSE)"]
results[numeric] = results[numeric].map(lambda x: f"{x:.4f}")
print(results.to_string(index=False))


plt.figure(figsize=(7, 5))
plt.plot(degrees, tr_err, marker='o', label="Training Error (MSE)",
   color='deeppink')
plt.plot(degrees, val_err, marker='s', label="Validation Error (MSE)",
   color='slateblue')

plt.xlabel("Degree")
plt.ylabel("Error")
plt.title("Bias-Variance Tradeoff")
plt.yscale("log")
plt.legend()
plt.grid(True)
plt.show()
```

Output

Degree	Training Error	Validation Error	Training Error (RMSE)	Validation Error (RMSE)
1	22.5455	16.0965	4.7482	4.0120
2	4.6603	12.3813	2.1588	3.5187
3	0.0000	2131.3515	0.0000	46.1666
4	0.0000	191.3448	0.0000	13.8327
5	0.0000	138.0542	0.0000	11.7496



Result

Successfully investigated the bias-variance tradeoff using polynomial regression on the Boston Housing dataset.